

INTERNAL BUILD REPORT

MTSAi Traffic Data Lake

What was built, how it works, and what's been verified in the Test environment

Reference document	TDR-004 / DL-PLAN-001, "MTSAi Traffic Data Lake" brief v0.1
Scope of this report	Phase 0, Phase 1 & Phase 2 (v1 slice), Test account only
Target AWS account	Test — 690293068614 (ap-south-1)
Repository	mtsai-datalake
Report date	15 September 2026

Phase 1 — Applied & Verified (Test)

Phase 2 — v1 Slice Built & Verified

Phase 0 — Blocked on DB access

Dev — Out of scope

§ Contents

1.	Summary
2.	Target Architecture
3.	Data Zones & Storage Layout
4.	Consumer Isolation Model
5.	Export Job Pipeline (Phase 2 v1 Slice)
6.	CI/CD & Deployment Pipeline
7.	What Has Actually Been Built
8.	Verification Results — Isolation
9.	Bugs Found & Fixed
10.	Outstanding Items

1 Summary

MTSAI's traffic data (camera events, trip events, third-party feeds) lives in a single Postgres database (RDS) that also runs the live production application. Analytical queries over that data compete with production traffic, are expensive to keep in RDS long-term, and several teams (analytics, forecasting, dashboards, auditors) all need access without touching the primary database directly. This project builds a minimal, serverless data lake on AWS S3 to solve that — storing cold data in an open format, queryable via SQL, with per-team access isolation and a path to per-record deletion for compliance (DPDPA/GDPR/PIPEDA).

This report covers what has actually been **built and verified** in the Test AWS account, not just what's planned. Everything below reflects real infrastructure that exists today and has been tested with live AWS API calls — not a design document.

Since the previous version of this report (14 September), the export job itself — the piece that actually moves data from Postgres into the lake — went from a placeholder to a working, tested pipeline (§5), and five more real bugs were found and fixed by running things instead of just reading the code (§9).

2 Target Architecture

All components are AWS-managed and serverless except the nightly export job, which runs as a scheduled container. Data flows one direction: from the operational database, into cold storage, out through a SQL query layer to each consumer.

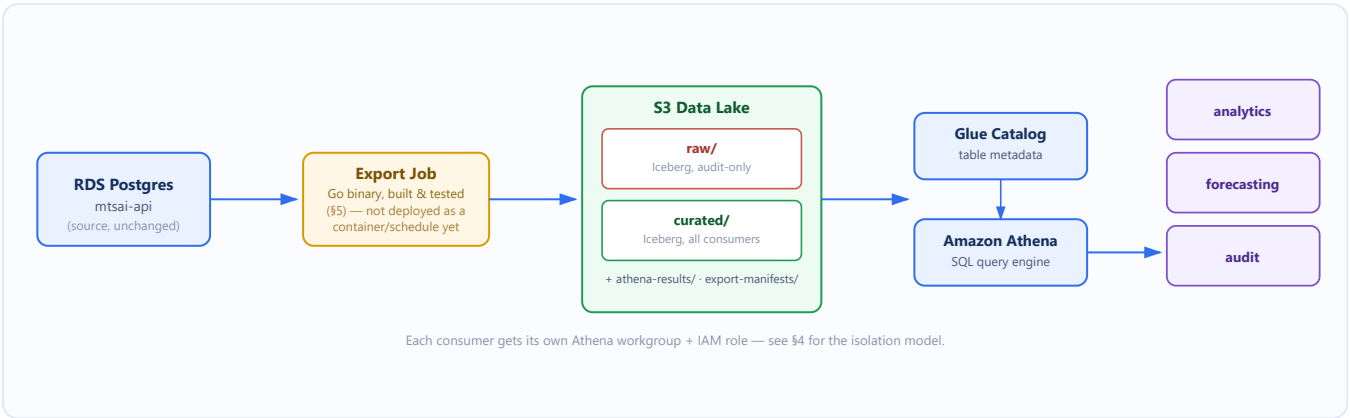


Figure 1 — end-to-end data flow, source to consumer.

3 Data Zones & Storage Layout

The lake uses two zones only, per bucket `mtsai-datalake-test-690293068614-ap-south-1` :

Zone	Purpose	Who can read it	Format
<code>raw/</code>	Faithful, unmodified copy of source rows. Audit/replay layer.	audit workgroup only	Apache Iceberg
<code>curated/</code>	Cleaned, deduplicated, analytics-ready tables.	All consumers (analytics, forecasting, audit)	Apache Iceberg
<code>athena-results/<consumer>/</code>	Per-consumer query output, 7-day expiry.	Only that consumer	CSV/metadata
<code>export-manifests/</code>	Reconciliation & audit trail for each export run.	Export job only (Phase 2)	JSON

Apache Iceberg (not plain Parquet folders) was chosen specifically because it supports row-level deletes — required for per-person erasure requests — plus schema evolution and time travel, without locking the lake into one query engine.

4 Consumer Isolation Model

Every consumer gets its own Athena workgroup *and* its own IAM role — never a shared identity. Access is enforced by IAM policy on the underlying S3 prefixes and Glue catalog resources, not by which workgroup happens to be selected in a console.

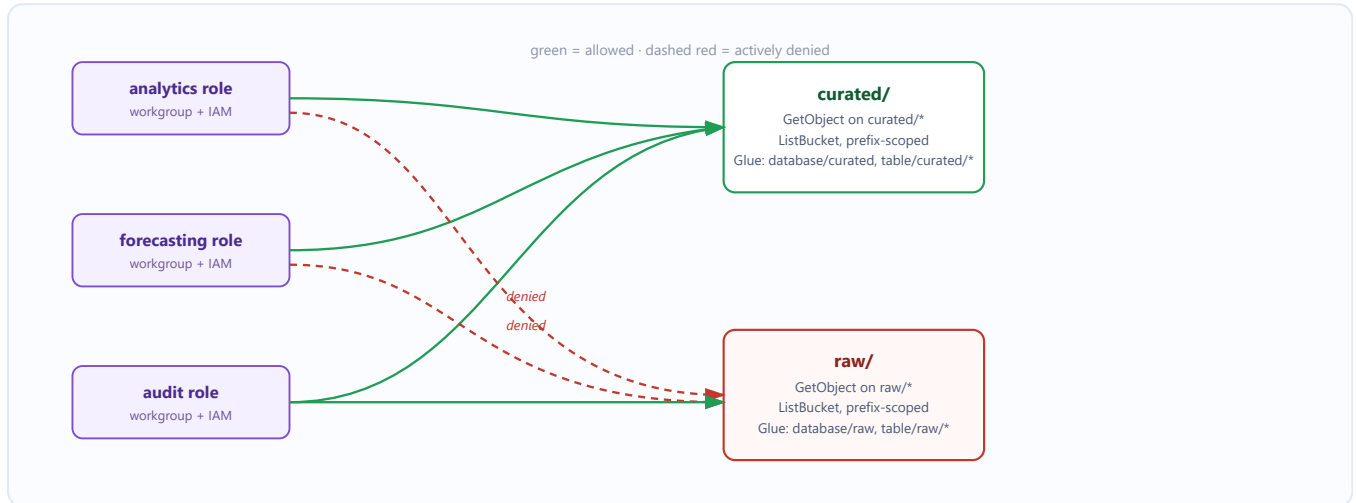


Figure 2 — verified access matrix: only **audit** can reach **raw/** ; all three reach **curated/** .

Why this needed three separate IAM layers, not one

Isolation had to be correct at every layer a query actually touches — S3 storage, the Glue metadata catalog, and Athena's own bucket-verification step — because a gap in any single layer breaks the whole guarantee. This was discovered empirically, not designed up front (see §9).

5 Export Job Pipeline (Phase 2 v1 Slice)

The export job (`export/` , Go module `mtsai-datalake-export`) is what actually moves data from Postgres into the lake — everything in §2-§4 is the destination and access-control layer around it, but until this existed nothing populated `raw/` for real. This v1 slice handles exactly **one table** — a synthetic fixture, `trip_events` , standing in for a real `mtsai-api` table until Phase 0 discovery identifies one (still blocked on Postgres access). Real `mtsai-api` access still doesn't exist, so it was tested against a local Docker Postgres container seeded with fixture rows — matching the brief's own stated testing convention (Section 8) — while every AWS-side call (S3, Glue, Athena) hit the real Test account.



Figure 1b — the export job's internal pipeline. Idempotency comes from step 2 (DELETE) before step 3 (INSERT) on every run, keyed by (table, event_date).

Check	Result
Read 5 fixture rows from Postgres, filtered by <code>event_date</code>	5 rows read (6th row, different date, correctly excluded)
Upload Parquet to S3 staging prefix	Uploaded, confirmed via <code>s3 ls</code>
Commit into <code>trip_events</code> Iceberg table via Athena	5 rows landed, values and <code>event_date</code> correct on re-query
Reconciliation (Postgres count/checksum vs. lake count/checksum)	Matched (checksum 15 = sum of trip_id 1..5)
Manifest written to <code>export-manifests/2026-01-15/trip_events.json</code>	Present, correct row count/checksum
Re-run for the same date (idempotency)	Still 5 rows, not 10 — confirmed independently via a fresh <code>COUNT(*)</code>
<code>docker build --platform linux/arm64</code>	Succeeds (image not pushed/deployed anywhere yet)

Explicitly out of scope for this slice (see `export/README.md`): pushing the image to ECR / wiring it into the existing ECS task definition, the real EventBridge nightly schedule actually triggering it (still blocked on VPC subnet IDs), backfill mode, curated-layer CTAS automation, and handling more than one table.

6 CI/CD & Deployment Pipeline

Infrastructure is provisioned via Terraform, applied through GitHub Actions using short-lived OIDC credentials — no long-lived AWS keys stored anywhere in CI.

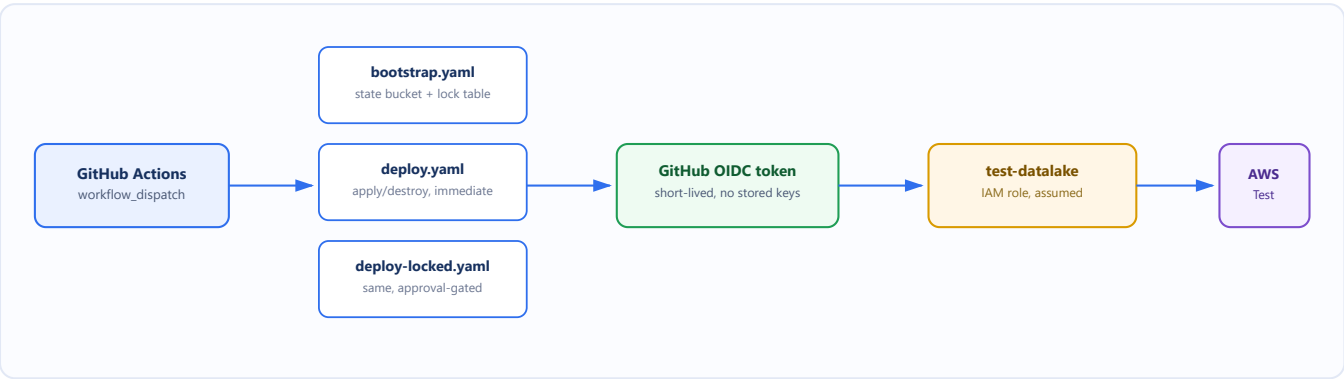


Figure 3 — deployment flow. Both deploy workflows share identical steps; only the GitHub Environment gate differs.

Workflow	Trigger	Approval gate	Purpose
bootstrap.yaml	Manual	None	Creates the Terraform state S3 bucket + DynamoDB lock table, once per account
deploy.yaml	Manual	None	Runs terraform apply or destroy immediately
deploy-locked.yaml	Manual	GitHub Environment reviewer	Same as above, but pauses for approval — used for anything destructive
debug-oidc.yaml	Manual	None	Decodes the OIDC token's claims to debug an IAM trust policy, without calling AWS

7 What Has Actually Been Built

Phase	Status	Notes
Phase 0 — Discovery	Blocked	No Postgres (mtsai-api) DB access from this environment yet; templates ready to fill in once access exists.
Phase 1 — Foundation	Applied & verified (Test)	S3 bucket, KMS key, Glue catalog, 3 Athena workgroups + IAM roles, ECS scaffolding, alarms — all live in Test.
Phase 2 — Export pipeline	v1 slice built & verified	Go export job works end-to-end for one table (\$5); not yet containerized/deployed for real.
Phase 3 — Governance & erasure	Not started	Erasure runbook drafted, not yet rehearsed.
Phase 4 — Postgres trim	Not started	Depends on Phase 2/3.

Live/working in Test (690293068614)

S3 lake bucket (+ separate SSE-S3 logs bucket) · KMS key · Glue `test_raw` / `test_curated` databases (now including a real `trip_events` Iceberg table) · 3 Athena workgroups with scoped IAM roles · ECS cluster + task definition (placeholder image) · SNS/CloudWatch failure alerting · the export job, run manually

Deliberately not built yet

Dev environment (out of scope per direction) · export image pushed to ECR / wired into ECS · VPC subnets for the nightly schedule · Postgres secret · production promotion · handling a second table

8 Verification Results — Isolation

Isolation was verified by actually assuming each consumer's IAM role via STS and running real Athena queries — not by switching workgroups while signed in as an admin, which doesn't test anything (the workgroup selection doesn't change which identity authorizes the query).

Role	Query against <code>curated</code>	Query against <code>raw</code>	Expected raw access?	Result
<code>analytics</code>	SUCCEEDED	FAILED (AccessDenied)	No	Correct
<code>forecasting</code>	SUCCEEDED	FAILED (AccessDenied)	No	Correct
<code>audit</code>	SUCCEEDED	SUCCEEDED	Yes	Correct

Repeatable via `runbooks/isolation-check.ps1`, committed to the repo — assumes all three roles automatically and reports pass/fail against expectations.

9 Bugs Found & Fixed

Eleven real bugs found across this project so far, none caught by `terraform plan`, `go build`, or code review — all found only by actually exercising the deployed system with real API calls.

Platform / Phase 1 (isolation and storage)

1. **S3 key enumeration leak.** `s3:ListBucket` was granted on the bare bucket ARN alongside prefix-scoped `s3:GetObject`. Since `ListBucket` is a bucket-level action, this let every consumer list (though not read) keys under `raw/` regardless of their actual read permissions. Fixed by scoping `ListBucket` with an `s3:prefix` condition per zone.
2. **Missing `s3:GetBucketLocation`.** Athena calls this on the results bucket before running *any* query, for every consumer. It was never granted anywhere, so every query failed immediately with "Unable to verify/create output bucket."
3. **Tag-based Glue permissions don't work for hand-created tables.** Table reads were conditioned on an `Environment` resource tag, but tables are created by hand via Athena DDL/CTAS, not by Terraform — they never receive that tag. This silently denied `glue:GetTable`, which Athena confusingly surfaced as `TABLE_NOT_FOUND` rather than an access error. Fixed by scoping Glue permissions with explicit ARNs (`database/<name>`, `table/<name>/*`) instead of tags — this also properly isolates raw-zone *metadata* by zone, which the tag approach never did.
4. **Glue's hierarchical authorization.** Reading a table requires an explicit Allow at every level of catalog → database → table, not just the most specific ARN. `GetDatabase` was granted at the catalog level but `GetTable` / `GetTables` / `GetPartition(s)` were not — blocking legitimate curated-zone reads even after every other permission was correct.
5. **S3 access logging silently did nothing.** The lake bucket logged to itself, but S3 server access logging doesn't support an SSE-KMS-encrypted destination (only SSE-S3) — the lake bucket's default encryption is SSE-KMS. `terraform apply` succeeded; no log objects were ever delivered. Fixed with a separate, SSE-S3 logs bucket as the target.
6. **...and that logs bucket had no delivery permission either.** Since April 2023, new S3 buckets default to Object Ownership "Bucket owner enforced" (ACLs disabled) — the classic ACL-based grant to the S3 Log Delivery group can't apply, and no bucket policy existed to grant it another way. Without either, delivery isn't delayed, it's permanently blocked. Fixed with an explicit bucket policy granting `logging.s3.amazonaws.com` write access, scoped to the lake bucket via `aws:SourceArn` / `aws:SourceAccount`. Log delivery confirmed working afterward (~5.5 hour observed latency, consistent with AWS's "best effort" description).

Export Job / Phase 2

1. **Athena's `WITH (...)` table syntax is CTAS-only.** `CREATE TABLE t (cols) WITH (external_location = ..., format = ...)` without an `AS SELECT` is rejected outright. Registering existing S3 files as a table needs classic Hive DDL instead: `CREATE EXTERNAL TABLE ... STORED AS PARQUET LOCATION ...`.
2. **`IF NOT EXISTS` breaks with `EXTERNAL`.** That same Hive DDL is rejected by the parser when `IF NOT EXISTS` is added — undocumented, confirmed empirically. Dropped it; each staging table name already has a unique run-ID suffix.
3. **`parquet-go` mis-encodes dates.** The library's reflection-based `time.Time` writer only correctly handles the Timestamp logical type (physical `int64`); for Date (physical `int32`) it still writes a raw nanosecond `int64` into the `int32` slot. `event_date` round-tripped as `-1454296-02-29` instead of `2026-01-15`. Root-caused by reading the library's source directly, then fixed by encoding the date as days-since-epoch by hand.

4. **Export-task role had nowhere to write Athena results.** No S3 permission existed for `athena-results/export/*` — every query the role runs would have failed.
5. **Export-task role couldn't clean up after itself.** No `glue:DeleteTable` permission for dropping the throwaway staging table each run creates.

All eleven fixes are applied and re-verified in Test as of this report (see §5 and §8).

10 Outstanding Items

Item	Why it's not done
Push export image to ECR / wire into ECS task	Phase 2 v1 slice runs as a local binary against Test AWS resources; not yet containerized-and-deployed for real.
Nightly export schedule	Needs real VPC subnet/security-group IDs for Test — not yet confirmed. Schedule is deliberately skipped (not failing) until then.
Postgres read access	No connection details or DB client available yet — blocks Phase 0 discovery entirely, and means the export job has only ever run against a synthetic fixture.
Second table / schema-generic export	v1 slice hardcodes one table (<code>trip_events</code>); config file is real but not yet driving Parquet schema generation.
AWS Budgets / cost alerts	Needs an alarm email and confirmation that cost-allocation tags are active org-wide.
Plan-on-PR / apply-on-merge CI	Current workflows are manual-dispatch only; automatic PR-triggered plans aren't built yet.
KMS key policy restriction (Section 6)	Currently relies on IAM alone rather than a resource-level key policy limited to specific roles.